

Travel Planner AI Agent - LangGraph Assignment

Overview

In this assignment, you will build a Travel Planner AI agent using the LangGraph library. The agent will help users plan a trip by understanding their preferences, suggesting destinations, creating itineraries, and addressing follow-up questions.

This assignment is designed to test your ability to:

- Build a multi-step reasoning workflow using LangGraph
- Implement state management and conditional logic
- Connect to external tools and APIs
- Handle user interactions effectively

Requirements

Core Functionality

Your Travel Planner agent should be able to:

1. **Understand user preferences:** Extract travel preferences (budget, duration, interests, etc.) from user input
2. **Suggest destinations:** Recommend appropriate destinations based on preferences
3. **Create itineraries:** Generate day-by-day travel plans
4. **Answer follow-up questions:** Respond to questions about the suggested plan

Technical Requirements

1. Use LangGraph to create a directed graph with multiple nodes representing different stages of the planning process
2. Implement proper state management to track user preferences and planning progress
3. Handle at least two external tools:
 - A mock destination database (provided as JSON)
 - A weather API (can be mocked with sample data)
4. Implement error handling and graceful fallbacks
5. Include 3+ test cases with expected outputs

Project Structure

Your project should include:

```
travel_planner/  
├── main.py           # Entry point for the application  
├── agent/  
│   ├── graph.py     # LangGraph implementation  
│   ├── nodes/       # Node implementations  
│   └── tools/        # External tool connections
```

```
├── state.py          # State management
├── data/
│   └── destinations.json # Mock destination database
├── tests/
│   └── test_agent.py    # Test cases
└── README.md          # Documentation
```

Detailed Steps

Step 1: Setting up the LangGraph Framework

Create a LangGraph with the following nodes:

- **PreferenceExtractor**: Extract travel preferences from user input
- **DestinationFinder**: Find suitable destinations based on preferences
- **ItineraryCreator**: Create a detailed itinerary
- **FollowUpHandler**: Handle follow-up questions about the plan

Step 2: State Management

Design a state object that can track:

- User preferences
- Selected destinations
- Current itinerary
- Conversation history

Step 3: External Tool Integration

Connect to:

- Destination database: Filter destinations by criteria
- Weather API: Check weather forecasts for destinations

Step 4: Testing

Write test cases for:

- Basic preference extraction
- Destination recommendation
- Complete planning workflow
- Follow-up question handling

Sample Data

A simplified **destinations.json** file with the following structure will be provided:

```
[
  {
    "name": "Paris",
```

```
    "country": "France",
    "tags": ["romantic", "culture", "food"],
    "budget_level": "high",
    "ideal_duration": [3, 7],
    "best_seasons": ["spring", "fall"]
  },
  ...
]
```

Evaluation Criteria

Your submission will be evaluated based on:

1. **Functionality:** Does the agent work as expected?
2. **Code Quality:** Is the code well-structured and documented?
3. **LangGraph Implementation:** Proper use of LangGraph concepts
4. **Error Handling:** How well does the agent handle unexpected inputs?
5. **Creativity:** Additional features or improvements beyond basic requirements

Submission

Please submit:

1. Source code repository (GitHub/GitLab link)
2. Brief documentation explaining your approach
3. Instructions to run your application

Resources

- LangGraph Documentation: <https://github.com/langchain-ai/langgraph>
- Sample code for getting started (included below)

Getting Started Code

```
from typing import Dict, List, Any
from langgraph.graph import StateGraph
from langchain_openai import ChatOpenAI

# Define the state structure
class AgentState:
    preferences: Dict[str, Any]
    destinations: List[Dict[str, Any]]
    itinerary: Dict[str, Any]
    history: List[Dict[str, str]]

# Define key nodes
def extract_preferences(state):
    # Implementation here
    pass
```

```
def find_destinations(state):
    # Implementation here
    pass

def create_itinerary(state):
    # Implementation here
    pass

def handle_followup(state):
    # Implementation here
    pass

# Build graph
def build_travel_agent():
    workflow = StateGraph(AgentState)

    # Add nodes
    workflow.add_node("extract_preferences", extract_preferences)
    workflow.add_node("find_destinations", find_destinations)
    workflow.add_node("create_itinerary", create_itinerary)
    workflow.add_node("handle_followup", handle_followup)

    # Add edges
    workflow.add_edge("extract_preferences", "find_destinations")
    workflow.add_edge("find_destinations", "create_itinerary")
    workflow.add_conditional_edges(
        "create_itinerary",
        lambda state: "handle_followup" if state.get("is_followup") else
"END"
    )
    workflow.add_edge("handle_followup", "END")

    # Set entry point
    workflow.set_entry_point("extract_preferences")

    return workflow.compile()
```

Good luck!